

Trabajo Práctico Integrador: Ciclo de Vida y Despliegue Continuo de una API

1. Objetivos del Trabajo Práctico.....	1
2. Modalidad de Trabajo y Presentación.....	1
3. Consignas Técnicas.....	2
Fase 1: Desarrollo Base y Documentación.....	2
Fase 2: Gestión de Cambios y Versionado.....	2
Fase 3: Empaquetado y Entorno de Desarrollo (Docker).....	3
Fase 4: Automatización CI/CD (GitHub Actions).....	3
Fase 5: Observabilidad y Monitoreo.....	4
4. Estructura del Informe Técnico y Presentación.....	4
5. Criterios de Evaluación.....	5

1. Objetivos del Trabajo Práctico

El objetivo principal de este trabajo es aplicar la filosofía, metodologías y herramientas del movimiento **DevOps** en un ciclo de vida de desarrollo de software end-to-end.

A lo largo del proyecto, deberán evidenciar la puesta en práctica de:

- **The Three Ways (Las Tres Formas):** Acelerar el flujo de trabajo (Left-to-Right), ampliar los bucles de retroalimentación (Right-to-Left) y fomentar una cultura de experimentación y aprendizaje continuo.
- **Mecanismos de Stop-the-Line (Andon Cord):** Automatización de verificaciones en el pipeline para mitigar la propagación de errores hacia producción.
- **Principios Lean y Agilidad:** Entrega incremental en pequeños lotes, reducción de desperdicio (*muda*) y visibilidad total del estado del código.

2. Modalidad de Trabajo y Presentación

- **Formato:** Individual o en parejas (máximo 2 integrantes).
- **Evaluación:** La calificación de la materia es **individual** mediante la defensa oral del proyecto en clase.
- **Entregables:**

1. **Repositorio público en GitHub** con el código fuente, configuración de pipelines e infraestructura como código.
2. **Informe Técnico:** Documento en el repositorio (formato **README.md** o PDF adjunto) que detalle las decisiones arquitectónicas, diagramas de flujo del pipeline, captura de las herramientas utilizadas y el análisis del proyecto bajo los principios Lean/DevOps.
3. **Presentación Sincrónica:** Exposición en clase mostrando el funcionamiento en vivo de la API, la ejecución del pipeline y el monitoreo de la aplicación.

3. Consignas Técnicas

Fase 1: Desarrollo Base y Documentación

1. **API REST:** Desarrollar una API básica (agnóstica en el lenguaje/framework de su elección). La lógica de negocio no es el foco de evaluación.
2. **Pruebas Unitarias:** Implementar una suite básica de pruebas automáticas para validar el funcionamiento mínimo de la API.
3. **Documentación Abierta:** La API **debe** incluir su interfaz de documentación interactiva accesible y expuesta (ej. Swagger, OpenAPI o equivalente nativo del framework). *Este punto es un requisito obligatorio de validación.*

Fase 2: Gestión de Cambios y Versionado

1. **Conventional Commits:** Todo el historial de commits del repositorio debe seguir la convención de *Conventional Commits* (**feat**:, **fix**:, **docs**:, **ci**:, etc.).
2. **Estrategia de Branching y Pull Requests:** Prohibido subir cambios directamente a la rama principal (**main/master**). Todas las modificaciones deben integrarse mediante **Pull Requests (PR)** debidamente documentados, incluyendo descripción del cambio y evidencia de pruebas ejecutadas.
3. **Estrategia de Versionado:** Implementar una estrategia de etiquetado y trazabilidad de artefactos seleccionando uno de los siguientes esquemas válidos para el contexto del proyecto:
 - **Semantic Versioning (SemVer):** Etiquetado mediante versiones semánticas estándar (**MAJOR.MINOR.PATCH**, ej: **v1.0.0**).
 - **Commit SHA / Git Hash:** Etiquetado inmutable basado en el hash corto del commit (**git rev-parse --short HEAD**, ej: **sha-a1b2c3d**).
 - **Versionado por Timestamp / Build Number:** Etiquetado basado en la fecha/hora de compilación o número incremental de ejecución del pipeline (ej: **2026.08.29-build.42**).

Fase 3: Empaquetado y Entorno de Desarrollo (Docker)

1. Dockerfile Optimizado:

- Implementar **Multi-stage build** para separar el entorno de compilación/dependencias del entorno final de ejecución.
- Utilizar una **imagen base ligera y específica** (ej: `alpine`, `slim`, `distroless`). Queda prohibido el uso del tag `latest`.
- Configurar un usuario sin privilegios elevados (**Non-root user**) para ejecutar la aplicación dentro del contenedor.
- Estructurar las instrucciones del Dockerfile ordenando las capas para **maximizar el uso del caché de Docker** (separar la descarga de dependencias de la copia del código fuente).

2. **Orquestación Local:** Crear un archivo `docker-compose.yml` funcional que permita levantar el entorno completo de la aplicación de manera reproducible localmente con un solo comando.

Fase 4: Automatización CI/CD (GitHub Actions)

1. Integración Continua (CI):

- Diseñar un workflow modular en **GitHub Actions** que se ejecute ante eventos en los Pull Requests.
- Incluir etapas automatizadas de linter y ejecución de pruebas unitarias (*Andon Cord*: si una prueba falla, el PR no se integra).

2. Publicación de Artefactos:

- Automatizar el build de la imagen Docker en el pipeline y su publicación a un registro público (**Docker Hub**).
- La imagen subida a Docker Hub **debe estar etiquetada automáticamente con el tag de la versión SemVer** generada en la release.

3. Despliegue Continuo (CD - Desafío opcional/adicional):

- Configurar el despliegue automático a una plataforma gratuita (ej. Render u otra de su elección).
- El despliegue debe ser lanzado mediante un **Deploy Hook**, pasando como parámetro la versión/tag exacta de la imagen generada en el paso de CI.

Fase 5: Observabilidad y Monitoreo

1. **Logs Estructurados:** Formatear la salida de logs de la API para que emita eventos estructurados en formato **JSON** (debe incluir al menos **timestamp**, **level**, **path** y **status_code**).
2. **Plataforma de Monitoreo:** Conectar la aplicación a una plataforma de telemetría (Grafana Cloud, Datadog, New Relic o Sentry).
3. **Dashboard Personalizado:** Diseñar un tablero desde cero (sin el uso de plantillas preconfiguradas).
4. **Métricas de Aplicación (Golden Signals):** El dashboard debe reflejar visualmente las señales doradas a nivel HTTP/Red mediante tráfico generado sobre la API:
 - **Tráfico:** Peticiones por segundo / RPM.
 - **Latencia:** Tiempos de respuesta (ej. p50, p95).
 - **Errores:** Tasa o conteo de respuestas HTTP 4xx y 5xx.

4. Estructura del Informe Técnico y Presentación

El informe final y la exposición deberán incluir los siguientes apartados, recordando utilizar imágenes:

1. **Arquitectura del Pipeline**
 - Diagrama de Flujo: Representación visual del ciclo de vida del código desde el **git push** hasta la publicación del artefacto o despliegue.
 - Componentes: Identificación clara de las herramientas integradas en el flujo (linter, runner de CI, registro de imágenes, plataforma de hosting).
2. **Justificación Técnica y Decisiones de Diseño**
 - Optimización de Contenedores: Explicación del **Dockerfile** (elección de imagen base, estrategia de capas, compilación multietapa (*multi-stage*) y prácticas de seguridad como usuario no-root).
 - Estrategia de Integración: Reglas de branching elegidas (ej. Trunk-Based, GitFlow) y configuración de protección de ramas.
3. **Aplicación de la Filosofía DevOps**
 - Primera Forma (Flujo y Consistencia):
 - ¿Qué tareas manuales de empaquetado y verificación quedaron 100% automatizadas en el pipeline?
 - ¿Cómo se asegura que el entorno de ejecución del contenedor sea idéntico en desarrollo y en CI/CD?
 - Segunda Forma (Feedback Rápido y *Andon Cord*):

- Interrupción del Pipeline: ¿En qué puntos exactos el flujo de CI "corta el cable" (*Andon Cord*) impidiendo que un artefacto defectuoso avance?
- Visibilidad de Fallos: Si la API empieza a responder con errores HTTP 5xx o latencia alta, ¿qué herramienta de monitoreo lo detecta y mediante qué señal (log, métrica, alerta)?
- Tercera Forma (Aprendizaje y Simulación de Fallos):
 - Experimento de Falla Controlada: Describir un escenario donde rompieron la aplicación intencionalmente (ej. test unitario fallido, variable de entorno faltante) y explicar cómo el sistema de CI/CD o monitoreo reaccionó para alertar al equipo.
- 4. **Principios Lean (Reducción de Desperdicio)**
 - Identificación de 2 o 3 "desperdicios" comunes en desarrollo de software (ej. esperas por compilación manual, bugs detectados tarde en producción, configuración manual de servidores) y cómo la solución implementada los evita.

5. Criterios de Evaluación

El proyecto se evaluará mediante una rúbrica analítica individual que contempla el cumplimiento de los siguientes componentes técnicos sobre una escala de 0 a 10 puntos:

- **Documentación Abierta de la API:** Requisito de validación obligatorio (API Docs navegables).
- **Gestión de Cambios (20%):** Adopción de Conventional Commits, SemVer y calidad en la documentación de los Pull Requests.
- **Empaquetado y Docker (25%):** Implementación de Multi-stage, optimización de caché, imagen base específica, usuario non-root y ejecución mediante Docker Compose.
- **Automatización CI/CD (30%):** Modularidad de workflows en GitHub Actions, ejecución automatizada de pruebas, publicación etiquetada en Docker Hub y automatización del deploy hook.
- **Observabilidad (25%):** Configuración de logs estructurados en JSON, construcción de dashboards desde cero y gráficos activos para las Golden Signals.